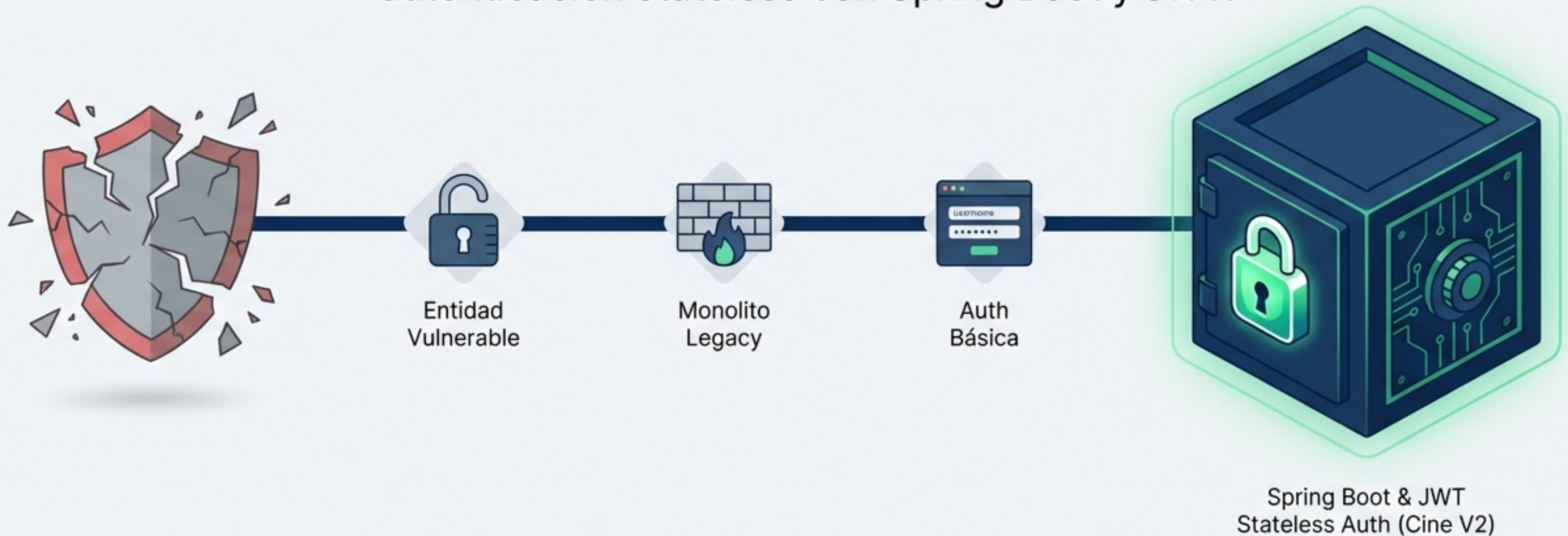
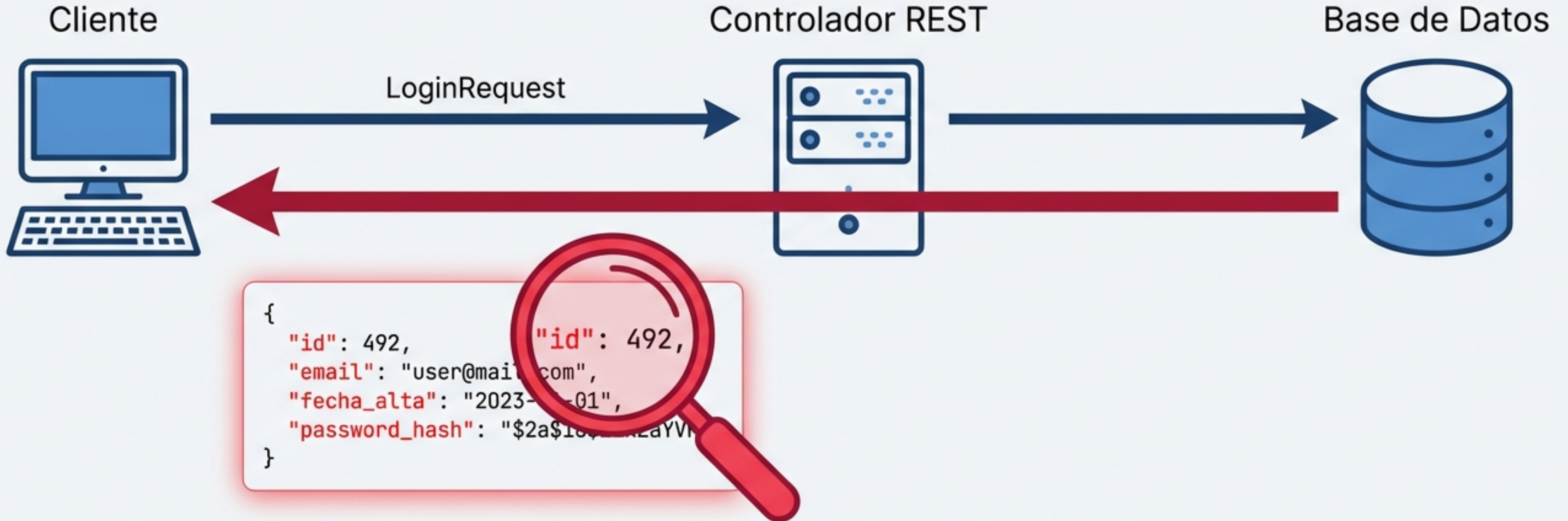


De Inseguro a Fort Knox: Arquitectura de Seguridad en APIs REST

Un viaje visual desde entidades vulnerables hasta autenticación stateless con Spring Boot y JWT.



FASE 1: La Regla de Oro y el Peligro de Exponer Entidades



Nunca se debe recibir ni devolver una Entidad JPA (clase mapeada a la base de datos) directamente en un Controlador REST.

- **El Problema:** Exponer entidades revela campos internos sensibles.
- **Sobrecarga:** Obliga a instanciar entidades completas en memoria innecesariamente.

La Solución: DTOs como Barrera Protectora



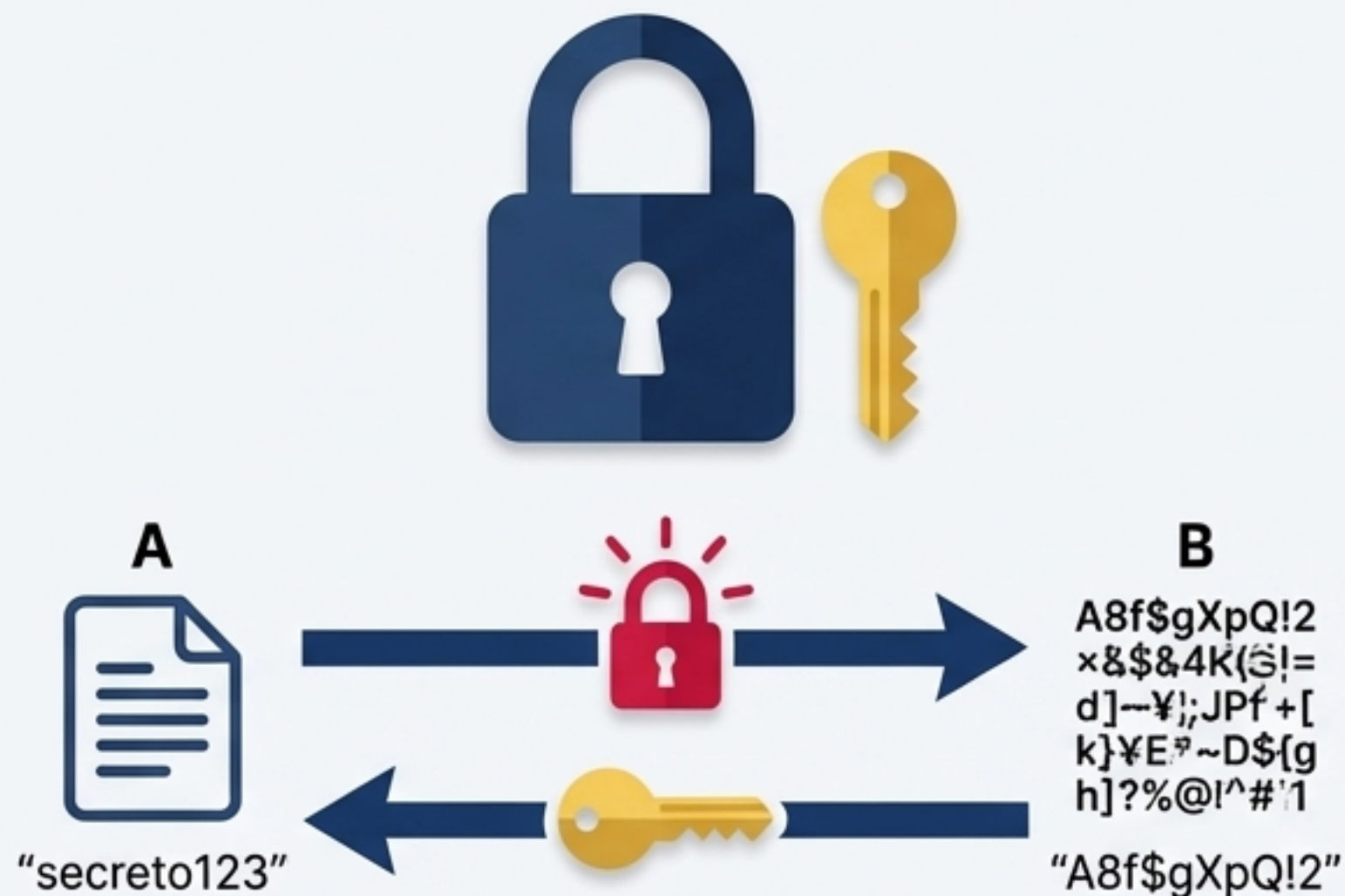
Let's define a

```
public record LoginRequestDTO(  
    String email, String password)  
{  
}  
  
public record LoginResponseDTO(String token) {}
```

- **Java Records (Java 16+):** Portadores de datos inmutables ideales para mover información sin lógica compleja.
- **Generación Automática:** En una sola línea de código, Java genera el constructor, accessors (getters), equals(), hashCode() y toString().
- **Inmutabilidad:** Los datos nacen y mueren iguales por defecto.

FASE 2: Protegiendo la Base de Datos (Cifrado vs. Hashing)

Cifrado - Malo para contraseñas



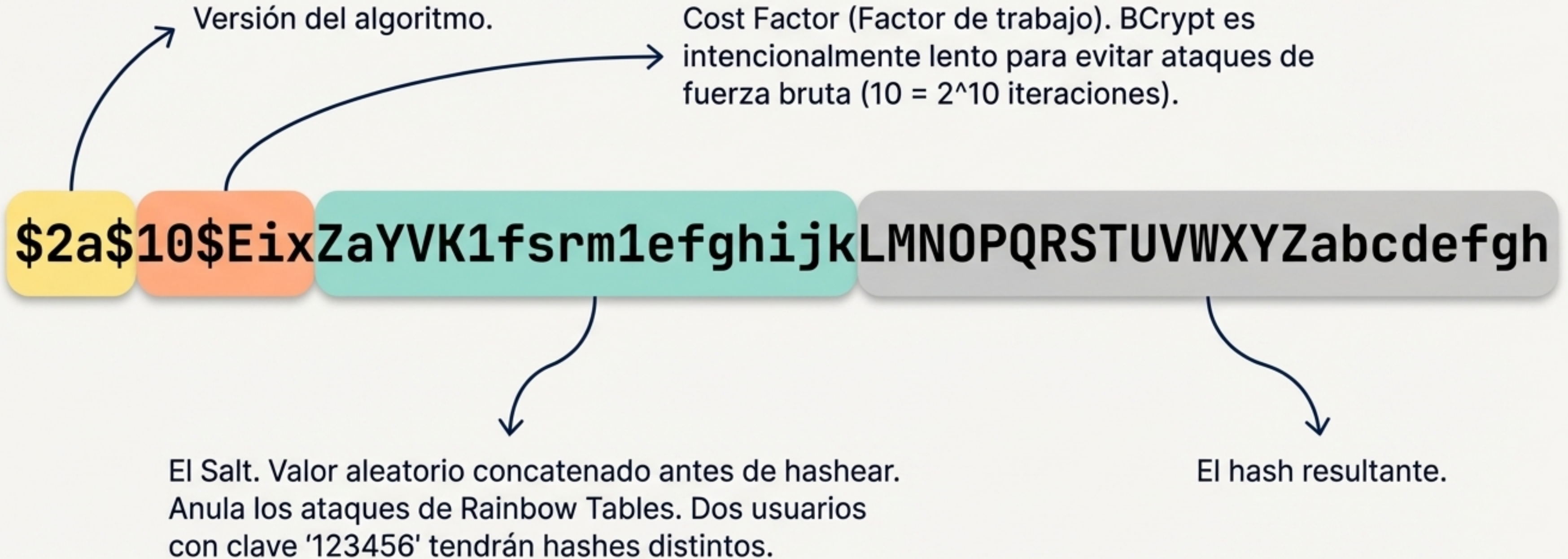
Bidireccional. Si un atacante roba la clave, descifra toda la base de datos.

Hashing - Correcto para contraseñas

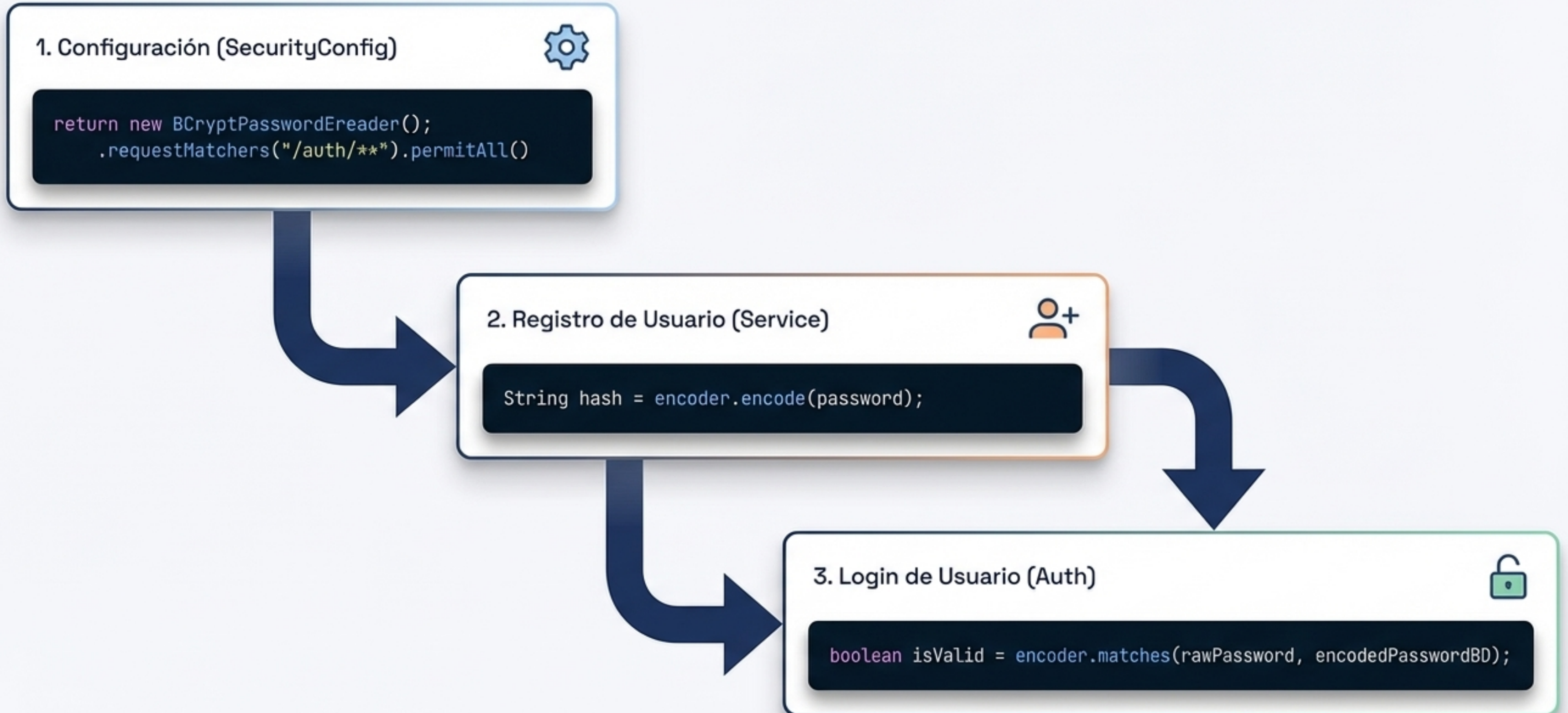


Unidireccional. Metáfora del batido de frutas: no tiene vuelta atrás. Para validar, se hasha el intento de login y se compara con el hash almacenado.

La Anatomía de un Hash BCrypt

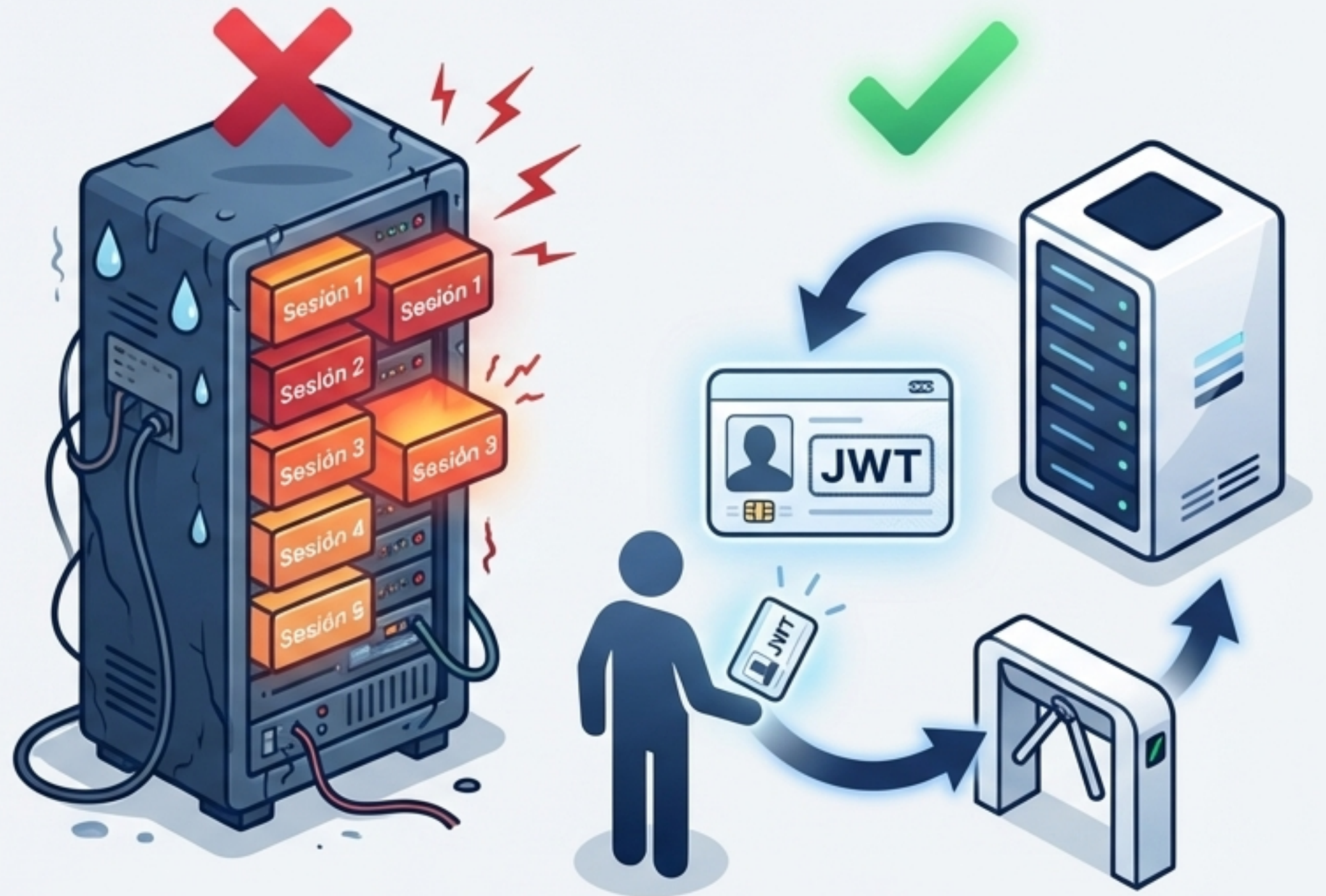


Implementación de BCrypt en Spring Boot



FASE 3: El Desafío del Estado y la Llegada de JWT

- **El Problema:** Mantener el estado (sesiones) en el servidor dificulta la escalabilidad (aplicaciones stateful).
- **La Solución:** Jason Web Token (JWT). Un estándar abierto (RFC 7519) para transferir información segura entre partes.
- **El Concepto:** Funciona como un Carné de Identidad Digital o pasaporte. El servidor lo emite una vez, y el cliente lo presenta en cada petición futura sin enviar credenciales.



La Anatomía de un JSON Web Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwiaWF0IjEwNjY3ODkwLCJpYXNzIjoiZm9udCJ9.KMUFsIDTnFmyG3nMiGM6H9FNFUR0f3wh7SmqJp-QV30

Header (Rojo): Indica el tipo de token (JWT) y el algoritmo de firma (ej. HS256).

Payload / Claims (Morado): Contiene los datos útiles del usuario (ej. email, roles, fecha de expiración).

Signature (Azul): El sello criptográfico que garantiza la integridad del token.

Header y Payload: Cuidado con tus Datos



ALERTA DE SEGURIDAD CRÍTICA



El Payload está codificado en Base64, **NO CIFRADO**.

```
{  
  "sub": "user@cine.com",  
  "role": "USER",  
  "exp": 1712000000  
}
```

- Cualquier persona que intercepte el token puede leer su contenido simplemente pegándolo en un decodificador.
- Regla de Oro: Nunca guardes contraseñas, números de tarjeta de crédito, o datos altamente sensibles dentro de los claims del JWT.

La Firma (Signature): El Sello de Seguridad

HMACSHA256 (Base64(Header) + "." + Base64(Payload), SECRETO)



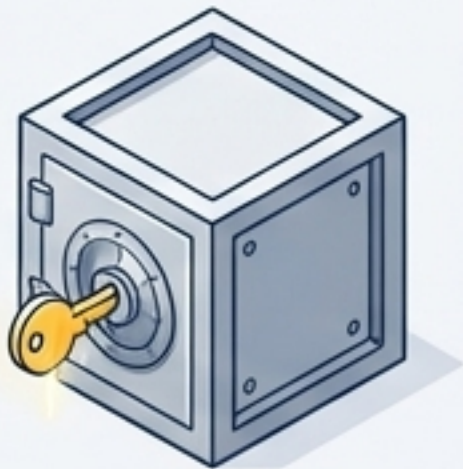
Garantía de Integridad: La firma asegura que el token no ha sido manipulado durante su transferencia. Si un atacante altera el Payload (ej. cambiándose el rol a ADMIN), la firma matemática fallará y el servidor lo rechazará.



El Secreto del Servidor: Solo el servidor conoce esta clave.



Buenas Prácticas: La clave secreta nunca debe ir escrita en el código fuente. Debe ser inyectada mediante Variables de Entorno (ej. `${JWT_SECRET}` en `application.properties`).



Inspección y Depuración: JWT.io

El Estándar de la Industria: jwt.io permite **decodificar**, verificar y generar tokens directamente en el navegador.

Seguridad Frontend: La validación y depuración ocurren 100% en el navegador (no transmite tokens al exterior).

Experimentación: Ideal para entender cómo los cambios en el Payload afectan la validez de la Firma al introducir el secreto correcto.

The screenshot shows the JWT.io web application interface. On the left, under the 'ENCODED' tab, a long Base64-encoded JWT token is displayed. On the right, under the 'DECODED' tab, the token's structure is shown in three sections: 'HEADER: ALGORITHM & TOKEN TYPE', 'PAYLOAD: DATA', and 'VERIFY SIGNATURE'. The header section shows a JSON object with 'alg' set to 'HS256' and 'typ' set to 'JWT'. The payload section shows a JSON object with 'sub' set to 'user@cinema.com', 'role' set to 'USER', and 'exp' set to 1712800000. The verify signature section shows a text input field containing 'your-256-bit-secret' and a preview of the signature calculation: `HNACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), secret )`.

```
eyJ0bGVKbnIIVGVJK6ZN3AATtUUKRVjdKSYPAHkaU6BjOV3wRPR2qLDVRANIZXInLYRUBYZANESaV7&NBoagYVhgK8ANTR2daEYzRA7AKQAKANmbLYZUKRIKIVeN6N0jEKZ0VEKYdGSCEAZJS2RASmV2G0NSXKSK2NBV2HoSSwKDIUOWXYNPK8TNALeQIOWVED608BI8KS7zY0VSAkvaN8GA66OnIKIvLlIQd2cm#jba0Va12GdUSH3vReV6ZSduzoNKVOKKB1N6ldeSs02c1xQERv7VEMw3TU000KINUMInamK0a4dU2Iww1e2y6gRpaRC1cYbdqEmfigcY0HwKIVX21A760871K50Re0R880N0NS1065K56AA0ZR1n38ITVS1SN3p1300o80469ANT0gVeK02BggOVwQSKMNU0IIXT3yoNuEvJVoNo9881EQw0.YNKNTB2SJAPdARIABaDILmGIKJARBYRarTGARESVCaN9DxENReg5NI2KDQ0IGIBwK9YAm0E9jRIE7BRFKKSAFHQAEPKVSAOLYAASG00CKAsUDPY2D0T2KVIXagKRYZBIIsNA
```

ENCODED 🔒

DECODED 🔓

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "sub": "user@cinema.com",  "role": "USER",  "exp": 1712800000}
```

VERIFY SIGNATURE

your-256-bit-secret 🔑

```
HNACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  secret  )
```


Generando el Token (JJWT y Spring Boot)



@PostConstruct: Transforma el secreto en un SecretKey criptográfico.

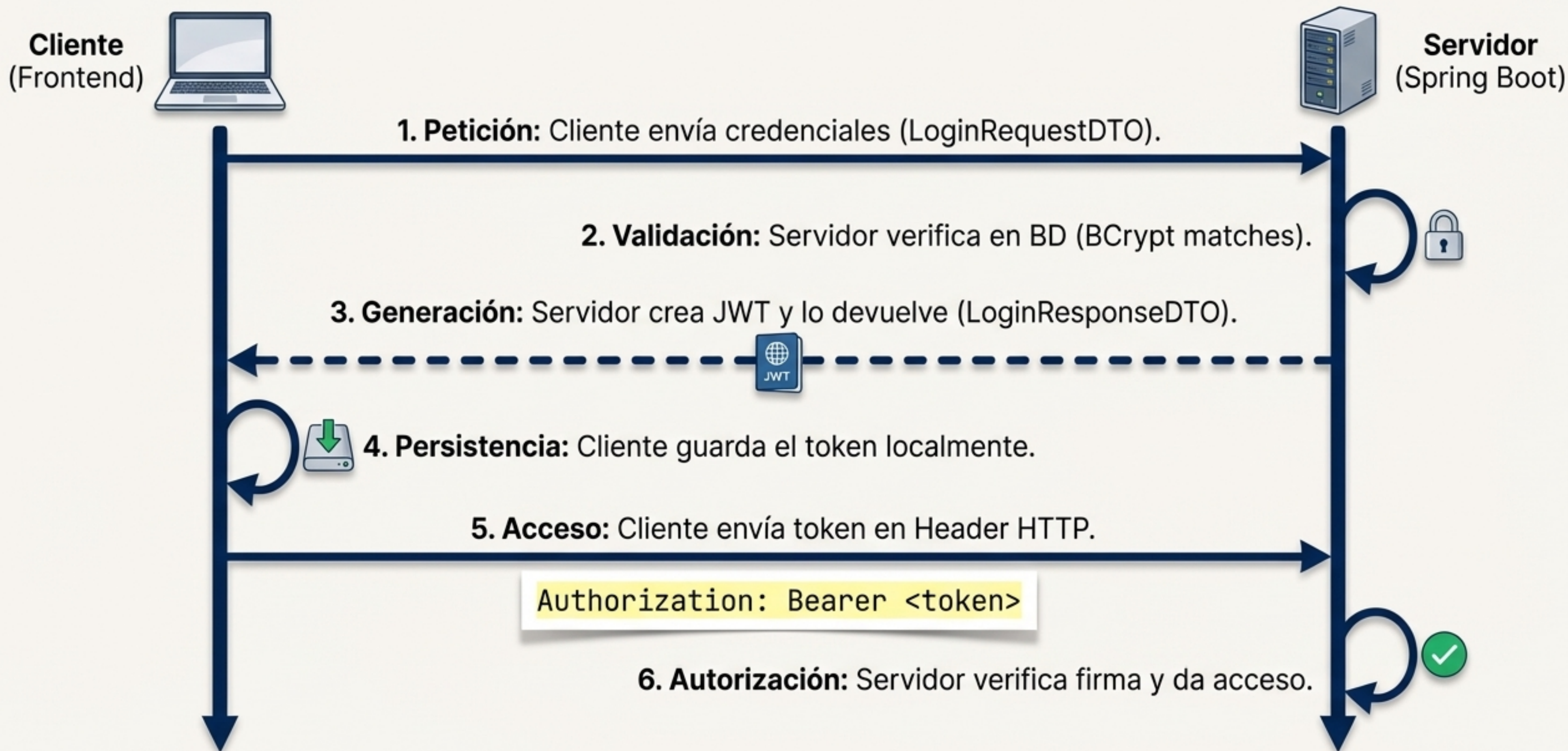
```
Jwts.builder()  
    .subject(email)  
    .claim("roles", roles)  
    .expiration(fecha)  
    .signWith(secretKey)  
    .compact();
```

Payload

Signature (Paso crítico)

Genera el String Base64

El Flujo de Autenticación Stateless



Integración Frontend: ¿Dónde guardar el Token?



SessionStorage (La Memoria RAM)

- ✓ La sesión muere al cerrar la pestaña o el navegador.
- ✓ Se limpia automáticamente.



LocalStorage (El Disco Rígido)

- ✓ Persiste hasta que se elimina manualmente.
- ✓ Sobrevive a cierres de pestaña y reinicios del navegador.



La Práctica Habitual: Se recomienda guardar el token en LocalStorage para mantener la sesión viva (persistencia) durante el tiempo de vida del token (ej. 1 día). Se elimina manualmente al ejecutar Logout.

Misión Cumplida: Arquitectura Fort Knox

- **Entrada (FASE 1):** Un escudo DTO intercepta los datos limpios.
- **Almacenamiento (FASE 2):** Base de datos protegida donde residen los hashes unidireccionales de BCrypt.
- **Comunicación (FASE 3):** Candado brillante (JWT) viajando stateless entre cliente y servidor.



- ✓ Sin exposición de entidades.
- ✓ Sin contraseñas en texto plano.
- ✓ Sin estado en el servidor.
- ✓ Una API REST moderna, escalable y segura.